Connecting to Two Roads to the GPU

Shared premise: explicit computation semantics can reduce graph-capture work.

The Python route

Python model -> capture and specialization -> computation graph -> optimization and code generation.

The graph-first route

Explicit graph -> optimization and code generation. Fusion, layouts and memory planning remain backend work.

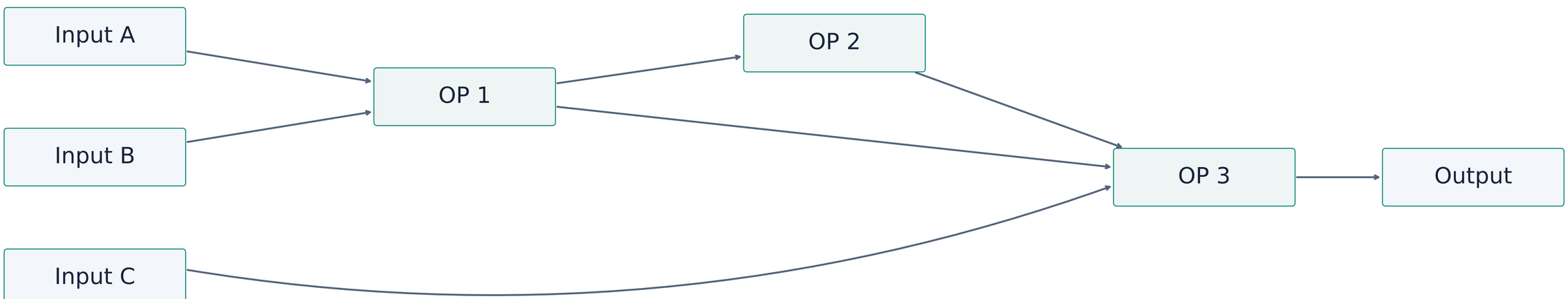
Our question

Direct FX and ONNX construction are also graph-first. What useful role would a Jac frontend add?

Context: professor-provided Two Roads to the GPU, pp. 1, 4-6. Paraphrase and discussion framing, not a new performance result.

FX and ONNX can be graph-first representations

A graph can be constructed directly; it does not have to be recovered from Python execution.



Explicit inputs, operation nodes, dependencies and shared intermediate values can all be represented in FX and ONNX.

Question: what does Jac add when constructing and using this same static graph?

Abstract dependency graph; no code shown. Static structure does not imply constant input values or fully fixed tensor shapes.

1. Linear

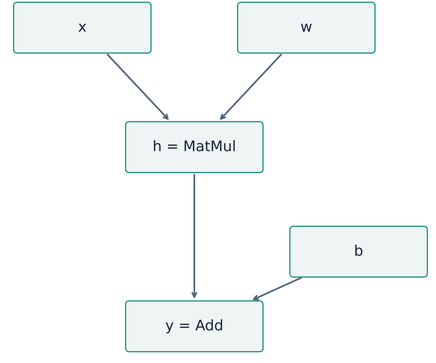
y = x @ w + b

	Jac: explicit nodes + edges	Python: FX node construction	Python: ONNX construction

1 Multiply	`h = MatMul();`
	`x +>: A() :+> h;`
	`w +>: B() :+> h;`
		`h = g.call_function(mm, (x, w))`	`nh = N("MatMul", ["x", "w"], ["h"])`

2 Add bias	`y = Add();`
	`h +>: A() :+> y;`
	`b +>: B() :+> y;`
		`y = g.call_function(add, (h, b))`	`ny = N("Add", ["h", "b"], ["y"])`

Computation graph



Shared h is computed once.

Semantic diagram; not a literal Jac or FX node export.

Aligned by operation semantics, not by source lines. Presentation excerpts only; runnable sources are unchanged.

Jac shorthand: A/B/X stand for typed input-port edges. Names, values, node/walker definitions and execution are omitted.

FX aliases: mm = torch.ops.aten.mm.default; add/relu are ATen operators. Input setup and execution are omitted.

Full sources and verification: jac/examples/graph_frontend_comparison/ | No timing or usability claim.

N = onnx.helper.make_node

G = onnx.helper.make_graph; types/setup omitted.

ONNX snippets: API examples, not runtime-tested.

2. Shared computation + residual

h = x @ w; y = relu(h) + h

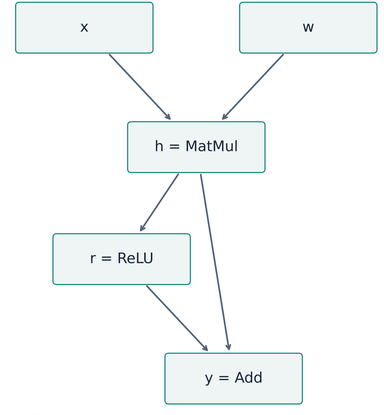
	Jac: explicit nodes + edges	Python: FX node construction	Python: ONNX construction

1 Multiply once	`h = MatMul();`
	`x +>: A() :+> h;`
	`w +>: B() :+> h;`
		`h = g.call_function(mm, (x, w))`	`nh = N("MatMul", ["x", "w"], ["h"])`

2 ReLU	`r = ReLU();` `h +>: X() :+> r;`	`r = g.call_function(relu, (h,))`	`nr = N("Relu", ["h"], ["r"])`

3 Residual add	`y = Add();`
	`r +>: A() :+> y;`
	`h +>: B() :+> y;`
		`y = g.call_function(add, (r, h))`	`ny = N("Add", ["r", "h"], ["y"])`

Computation graph



Shared h is computed once.

Semantic diagram; not a literal Jac or FX node export.

Aligned by operation semantics, not by source lines. Presentation excerpts only; runnable sources are unchanged.

Jac shorthand: A/B/X stand for typed input-port edges. Names, values, node/walker definitions and execution are omitted.

FX aliases: mm = torch.ops.aten.mm.default; add/relu are ATen operators. Input setup and execution are omitted.

Full sources and verification: jac/examples/graph_frontend_comparison/ | No timing or usability claim.

N = onnx.helper.make_node

G = onnx.helper.make_graph; types/setup omitted.

ONNX snippets: API examples, not runtime-tested.

3. Conditional: same result, different construction

h = x @ w; y = h + x if c else h - x

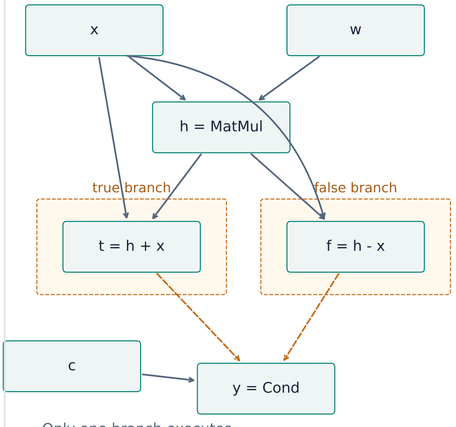
	Jac: explicit nodes + edges	Python: FX node construction	Python: ONNX construction

1 Shared multiply	`h = MatMul();`
	`x +>: A() :+> h;`
	`w +>: B() :+> h;`
		`h = g.call_function(mm, (x, w))`	`nh = N("MatMul", ["x", "w"], ["h"])`

2 Branch bodies	`t = Add(); f = Subtract();`
	`h +>: A() :+> t;`
	`x +>: B() :+> t;`
	`h +>: A() :+> f;`
	`x +>: B() :+> f;`
		`t, f = cond_modules(g, root, add, sub)`
		`# t(h,x) -> clone(h+x)`
		`# f(h,x) -> clone(h-x)`
			`tg = G([N("Add", ["h", "x"], ["t"])],`
			`"then", [], [t_info])`
			`fg = G([N("Sub", ["h", "x"], ["f"])],`
			`"else", [], [f_info])`

3 Select + join	`gate +>: Then() :+> t;`
	`gate +>: Else() :+> f;`
	`t +>: X() :+> y;`
	`f +>: X() :+> y;`
		`r = g.call_function(cond, (c, t, f, (h, x)))`
		`y = g.call_function(getitem, (r, 0))`
			`ny = N("If", ["c"], ["y"],`
			`then_branch=tg, else_branch=fg)`

Computation graph



Only one branch executes.
Dashed arrows: lazy result selection.

Semantic diagram; not a literal
Jac or FX node export.

Aligned by operation semantics, not by source lines. Presentation excerpts only; runnable sources are unchanged.

Jac shorthand: A/B/X stand for typed input-port edges. Names, values, node/walker definitions and execution are omitted.

Cond is not a one-to-one node mapping: Jac uses a gate + control edges + output join; FX uses two branch modules.

mm = torch.ops.aten.mm.default. Gate setup / wrapping omitted; custom Jac types are not builtins.

N = onnx.helper.make_node

G = onnx.helper.make_graph; types/setup omitted.

Outer h/x captured; only one branch runs.

What the examples show so far

All three representations express the math; graph-first benefits are not unique to Jac.

Linear and shared computation

FX handles, ONNX value names and native Jac edges encode dependencies and shared intermediate values.

Conditional selection

The native Jac example handwrites gate, routing and readiness behavior. FX uses Cond with branch GraphModules; ONNX uses If with branch GraphProto attributes.

Current limitation

The code does not establish that Jac is simpler. These custom native node/walker types are not automatically lowered to FX.

These are observations about the shown implementations, not a proof that either language is universally better.

Research goal to confirm

Which contribution should this frontend aim to demonstrate?

Programming experience

A better way to express an actual user task: compare against Python helpers and torch.cond, with equal abstraction levels.

Semantic guarantees

Specify additional shape, effect or control-flow guarantees, and show what errors or manual reasoning they eliminate.

Optimization or performance

Identify information or analysis missing from the baseline, then measure its effect. The same FX graph alone predicts no speed gain.

Question for discussion: If Jac generates the same FX graph, what evidence would make the extra frontend worth using?